SDN MINI PROJECT

SRN: PES2UG24CS323

NAME: NIRAV CHOUDHARY

SIMPLE QOS PRIORITY CONTROLLER

Repository: https://github.com/Nirav6769/SDN_QOS_Priority_Controller

ABSTRACT:

Software Defined Networking (SDN) is a modern networking approach that separates the control plane from the data plane, making network management more flexible and centralized. In this project, a Simple QoS Priority Controller is implemented using the Ryu SDN controller and the Mininet network emulator. The controller manages traffic by installing OpenFlow rules into the Open vSwitch switch based on packet type. ICMP traffic, such as ping packets, is given higher priority, while UDP traffic is assigned lower priority. This allows important traffic to be handled more efficiently. The controller dynamically updates the switch flow table whenever new traffic is detected. Experimental results show that the network topology was created successfully, devices were able to communicate correctly, and the priority rules were enforced as expected. The project demonstrates how SDN can be used to control traffic flow and improve Quality of Service (QoS) in a network.

TOOLS USED:

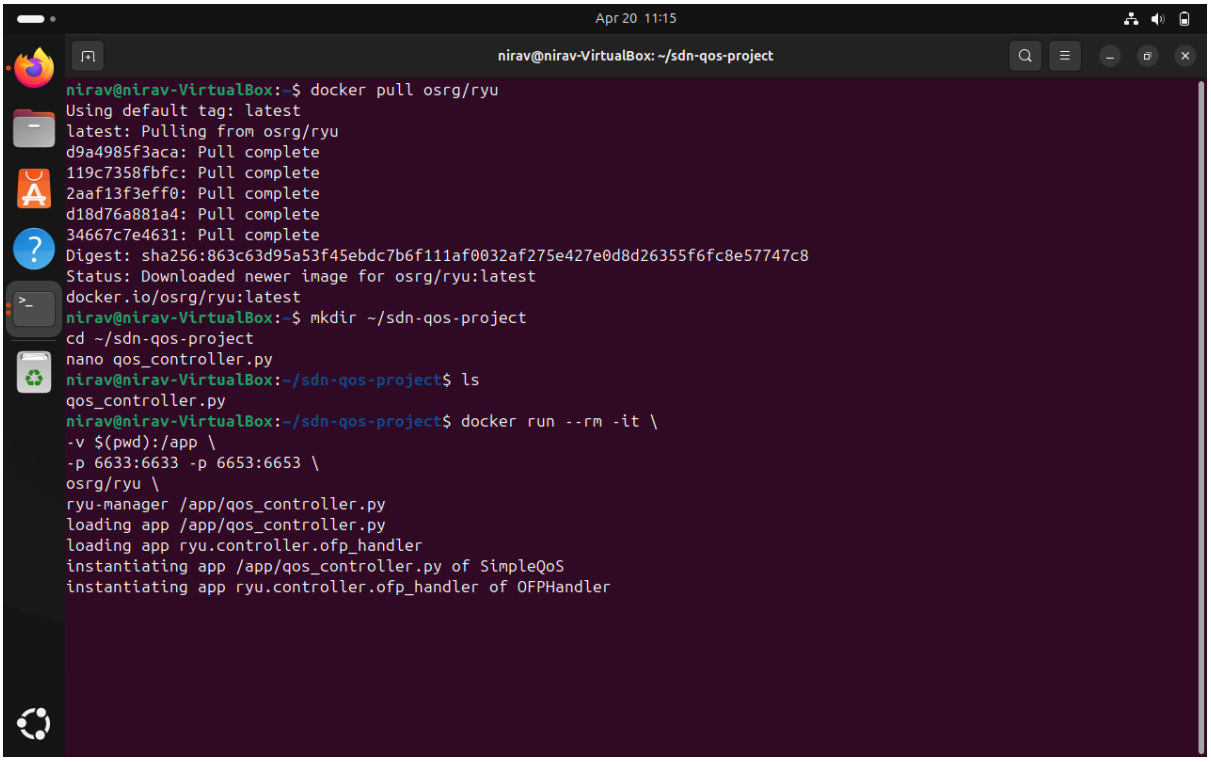
- Ubuntu Virtual Machine
- Mininet
- Ryu Controller
- Docker
- Open vSwitch

TOPOLOGY:

```
h1 --- s1 --- h2
        |
    Controller
```

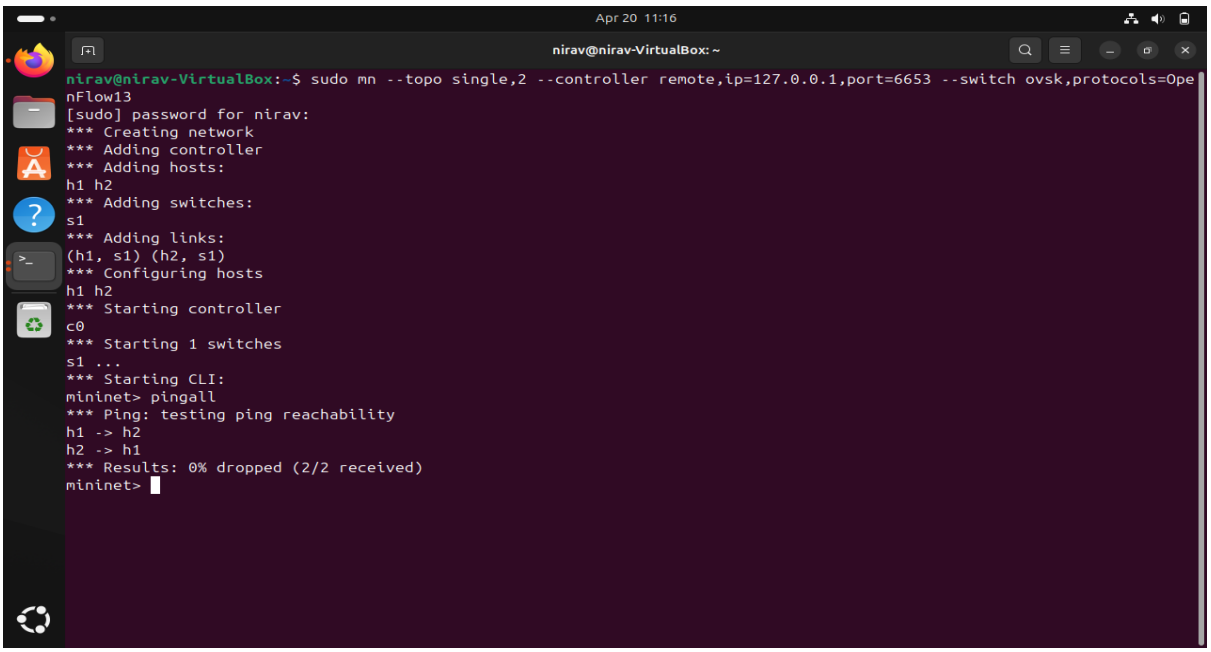
SCREENSHOTS:

1)Ryu Controller Initialization using Docker:



The Ryu SDN controller was deployed using Docker and the custom Python controller (qos_controller.py) was successfully loaded

2) Mininet Topology Creation and Connectivity Verification



A single-switch topology containing two hosts (h1, h2) and one OpenFlow switch (s1) was created using Mininet. The pingall command verified successful end-to-end communication with 0% packet loss.

3) OpenFlow Flow Table Installed in Switch s1

```
Apr 20 11:17
nirav@nirav-VirtualBox: ~
nirav@nirav-VirtualBox:~$ sudo ovs-ofctl -O OpenFlow13 dump-flows s1
[sudo] password for nirav:
 cookie=0x0, duration=163.026s, table=0, n_packets=4, n_bytes=392, priority=200,icmp actions=FLOOD
 cookie=0x0, duration=163.026s, table=0, n_packets=0, n_bytes=0, priority=10,udp actions=FLOOD
 cookie=0x0, duration=163.026s, table=0, n_packets=24, n_bytes=1720, priority=1 actions=FLOOD
nirav@nirav-VirtualBox:~$
```

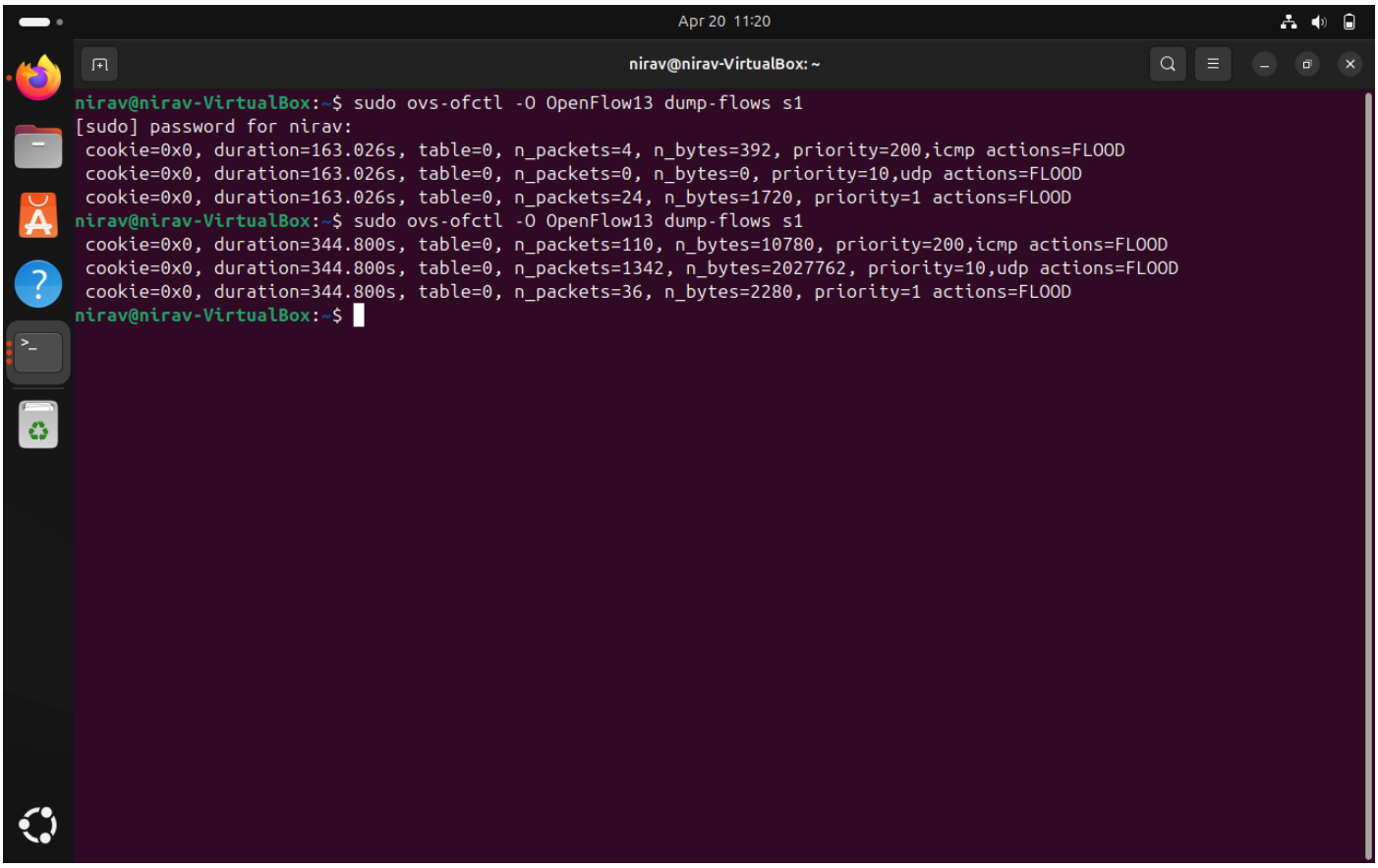
The Open vSwitch flow table was examined using ovs-ofctl dump-flows. The controller installed prioritized rules where ICMP traffic received priority 200, UDP traffic received lower priority 10, and unmatched traffic followed the default rule. Packet counters confirm traffic was matched successfully.

4) Simultaneous UDP Traffic and ICMP Ping Test

```
Apr 20 11:18
nirav@nirav-VirtualBox: ~
s1 ...
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
mininet> h2 iperf -s -u &
mininet> h1 iperf -c h2 -u -t 15 &
mininet> h1 ping h2
------------------------------------------------------------
Client connecting to 10.0.0.2, UDP port 5001
Sending 1470 byte datagrams, IPG target: 11215.21 us (kalman adjust)
UDP buffer size:  208 KByte (default)
------------------------------------------------------------
[  1] local 10.0.0.1 port 39404 connected with 10.0.0.2 port 5001
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.693 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.065 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.036 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.038 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.178 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.052 ms
[ ID] Interval       Transfer     Bandwidth
[  1] 0.0000-15.0175 sec  1.88 MBytes  1.05 Mbits/sec
[  1] Sent 1342 datagrams
[  1] Server Report:
[ ID] Interval       Transfer     Bandwidth        Jitter   Lost/Total Datagrams
[  1] 0.0000-15.0172 sec  1.88 MBytes  1.05 Mbits/sec   0.017 ms 0/1341 (0%)
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=0.067 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=0.059 ms
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=0.079 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=0.108 ms
```

UDP background traffic was generated between hosts using iperf while ICMP ping packets were transmitted simultaneously. The successful ping responses during UDP load demonstrate traffic handling and rule-based prioritization by the controller.

5) OpenFlow Flow Table with Active Packet Counters



The flow table of switch s1 was inspected before and after traffic transmission. Packet counters increased for ICMP and UDP rules, confirming that the controller-installed flow entries were actively matching network traffic.